NEXO

GESTIÓN DE TRABAJO

Informe de análisis

demo full-stack de lista de tareas

“Tu trabajo, en orden.”

Fecha de elaboración: 10 de noviembre de 2026

Evaluación técnica de la aplicación de gestión de tareas

Índice

1. Resumen ejecutivo 3
2. Clasificación 3
3. Evidencia revisada 3
4. Qué está funcionando bien 3
   4.1. Arquitectura simple y legible 3
   4.2. Persistencia local apropiada para una demo 3
   4.3. CRUD funcional en la práctica 3
   4.4. Validación básica en la API 4
   4.5. CORS configurado para la demo 4
5. Gaps o debilidades importantes 4
   5.1. Falta de pruebas automatizadas 4
   5.2. No hay autenticación ni autorización 4
   5.3. Estructura mínima y sin capas adicionales 4
   5.4. No hay migraciones ni estrategia de base de datos definida 4
   5.5. Falta de observabilidad y robustez operativa 4
   5.6. CORS rígido y acoplado al localhost 5
   5.7. Ausencia de CI/CD o automatización de calidad 5
6. Riesgos y puntos ciegos 5
   6.1. Riesgo de falsa sensación de “aplicación completa” 5
   6.2. Dependencia de un único origen y un único entorno local 5
   6.3. Datos sin control ni auditoría 5
   6.4. Validación mínima de negocio 5
7. Dirección y trayectoria 5
8. Veredicto final 5
9. Siguientes acciones recomendadas 6
   9.5. Separación de responsabilidades 6
10. Decisiones a tomar según el enfoque de la clase 6
   Enfoque simple 6
   Enfoque más completo 6
11. Qué evitar para no complicar demasiado la demo 6
12. Recomendación final 7
13. Propuesta de enunciado para la clase 7
14. Conclusión 7
15. Siguiente paso recomendado 7

1. Resumen ejecutivo

Este repositorio es una demo compacta y coherente de una aplicación full-stack para gestionar tareas. La arquitectura está claramente dividida en dos capas: un backend en ASP.NET Core con SQLite y un frontend en React + Vite + TypeScript. El proyecto cumple bien su propósito de ejemplo didáctico y de validación técnica de un CRUD básico, pero no parece orientado a producción ni a un producto empresarial completo.

La evidencia disponible sugiere que se trata de una demo/prototipo educativo con intención de demostrar integración entre API REST, Entity Framework Core, persistencia local y consumo desde un frontend moderno. La base es sólida para aprender arquitectura web básica, pero faltan elementos de madurez operativa, seguridad y automatización que sí esperaría un sistema con más alcance.

2. Clasificación

Clasificación: demo / prototipo educativo.

No hay señales de un producto de negocio completamente maduro: no hay autentificación, ni despliegue real, ni pruebas automatizadas, ni políticas de seguridad ni observabilidad. Todo apunta a una aplicación orientada a la enseñanza y la validación de conceptos técnicos, no a una aplicación de producción.

3. Evidencia revisada

Se revisaron los siguientes puntos del repositorio:

- README principal: describe la app como demo full-stack con .NET + SQLite + React + Vite.
- Backend: estructura en `TodoApi/` con `Program.cs`, `Controllers/TodosController.cs`, `Data/AppDbContext.cs`, `Models/TodoItem.cs`, DTOs y configuración SQLite.
- Frontend: `todo-ui/src/App.tsx` con lógica para listar, crear, modificar y borrar tareas.
- Configuración de CORS y de base de datos en `Program.cs` y `appsettings.json`.
- Documentación de análisis dentro de `docs/`.

4. Qué está funcionando bien

4.1. Arquitectura simple y legible

La separación entre backend y frontend es clara y fácil de seguir. El backend expone una API REST con endpoints CRUD y el frontend comunica con esa API mediante `fetch`.

4.2. Persistencia local apropiada para una demo

El uso de SQLite es una elección muy sensata para una aplicación de prueba o ejercicio académico. No requiere infraestructura extra ni configuración compleja, y permite que el proyecto funcione con pocos requisitos.

4.3. CRUD funcional en la práctica

Se observa una implementación completa de las operaciones básicas:

- listar tareas

- crear una tarea
- editar una tarea
- alternar estado completado
- eliminar una tarea

La API y la UI están alineadas con esas capacidades.

4.4. Validación básica en la API

Los DTOs de creación y actualización usan `DataAnnotations` para limitar el título y la descripción. También se valida que el título no esté vacío antes de crear o actualizar una tarea. Eso demuestra una capa inicial de control de entrada.

4.5. CORS configurado para la demo

La API tiene política CORS para aceptar peticiones desde `http://localhost:5173`, que es la URL típica del frontend React en desarrollo. Esto demuestra una integración real entre ambos lados del proyecto.

5. Gaps o debilidades importantes

5.1. Falta de pruebas automatizadas

No hay evidencia de tests unitarios, de integración o de UI. Para un proyecto de aprendizaje esto puede ser suficiente, pero para un nivel de madurez mayor sería una carencia crítica.

5.2. No hay autenticación ni autorización

La aplicación no tiene ningún mecanismo de seguridad para proteger la API ni distinguir usuarios. Esto es totalmente consistente con una demo, pero limita seriamente la aplicabilidad fuera del entorno de aprendizaje.

5.3. Estructura mínima y sin capas adicionales

Aunque la separación actual es razonable, no hay separación avanzada entre dominio, servicios, repositorios, validación de negocio ni arquitectura más escalable. Para un ejemplo de aprendizaje, esto está bien; para un proyecto que quiera crecer, sería un punto de refactorización.

5.4. No hay migraciones ni estrategia de base de datos definida

La base de datos se crea con `EnsureCreated()` en `Program.cs`. Eso es un patrón útil para demos rápidas, pero no es la estrategia más robusta para un proyecto con evolución real de esquema. No hay evidencia de migraciones de EF Core.

5.5. Falta de observabilidad y robustez operativa

No se observa:

- logging estructurado más allá del registro ASP.NET estándar
- health checks
- manejo de errores centralizado
- políticas de retry o resiliencia
- configuración por entorno diferenciada más allá de `Development`

5.6. CORS rígido y acoplado al localhost

La política CORS solo acepta un origen exacto (`http://localhost:5173`). Esto funciona para la demo en local, pero es una limitación clara si se quisiera desplegar la app o usar otros entornos.

5.7. Ausencia de CI/CD o automatización de calidad

No se observa configuración de GitHub Actions, linters ni validación automatizada. Para un proyecto que pretende ser mantenido por un equipo, esta ausencia es un riesgo real.

6. Riesgos y puntos ciegos

6.1. Riesgo de falsa sensación de “aplicación completa”

La app tiene un flujo CRUD funcional, pero ese valor no equivale a una solución productiva. El repositorio podría verse como “terminado” por su simplicidad, cuando en realidad solo cubre el caso de uso mínimo.

6.2. Dependencia de un único origen y un único entorno local

La app está diseñada para ejecutarse en localhost con un único frontend. Eso funciona bien para un laboratorio, pero limita la migración a entornos multiusuario o desplegados.

6.3. Datos sin control ni auditoría

No hay trazabilidad de quién creó, editó o eliminó tareas. Para un escenario más serio, sería necesario modelar usuarios, timestamps más completos y auditoría de cambios.

6.4. Validación mínima de negocio

La validación actual contempla título obligatorio y longitud máxima, pero no hay control de duplicados, prioridad, estados avanzados, fechas límite o reglas de negocio más complejas.

7. Dirección y trayectoria

La tendencia del proyecto es clara: enseñar un stack moderno y funcional en un contexto pequeño, sin introducir complejidad innecesaria. La dirección es sana si el objetivo es formación, demostración técnica y aprender patrones básicos full-stack.

La trayectoria no muestra drift ni desconexión con el objetivo: el repo sigue siendo una demo con una estructura prudente, y la tecnología elegida es coherente con ese propósito. Sin embargo, sí muestra una clara frontera entre “aprendizaje” y “sistema listo para usar en producción”.

8. Veredicto final

Este repositorio se lee como una demo educativa y un prototipo funcional, no como un producto listo para producción. Tiene una base técnica coherente y una implementación de CRUD muy clara, pero le faltan pruebas, seguridad, gobernanza de datos, automatización y una estrategia más madura de despliegue y evolución.

La dirección actual es razonable si el objetivo principal es demostrar una arquitectura full-stack con .NET y React. No es, en cambio, una base sólida para un sistema con requisitos de negocio reales sin más definición de alcance y endurecimiento técnico.

9. Siguientes acciones recomendadas

1. Añadir pruebas automatizadas de backend y frontend.
2. Introducir una estrategia de migración con Entity Framework Core en lugar de `EnsureCreated()`.
3. Definir un modelo de entorno (dev/staging/prod) y variables de configuración.
4. Mejorar la seguridad con autenticación y validación de acceso en la API.
5. Añadir observabilidad básica: logs, health checks y diagnóstico de errores.
6. Evaluar si la demo debe mantenerse como ejercicio didáctico o evolucionar hacia un MVP con requisitos y estructura más formal.
7. Considerar un README más explícito sobre el alcance real del proyecto para evitar interpretaciones erróneas de madurez.

En conclusión, el proyecto cumple muy bien su función como ejemplo de arquitectura full-stack simple y ejecutable, pero no debe interpretarse como una solución de nivel productivo sin más trabajo de diseño y hardening.

9.5. Separación de responsabilidades

- Controllers: manejar HTTP
- Services: lógica de negocio opcional
- Repository/Data: acceso a datos
- DTOs: contratos de entrada/salida
- Frontend: render y UX

10. Decisiones a tomar según el enfoque de la clase

Enfoque simple

Si el objetivo principal es enseñar CRUD y conexión con BD:

- backend con controllers
- EF Core + SQLite
- frontend con React y varios componentes básicos
- sin autenticación
- sin despliegue a Azure

Enfoque más completo

Si el objetivo es una demo más profesional:

- capas de servicio
- DTOs y validación
- estructura limpia por dominio
- test simple de API
- manejo de errores global
- diseño más pulido en interfaz

11. Qué evitar para no complicar demasiado la demo

- añadir autenticación en la primera iteración

- incluir demasiadas entidades o relaciones
- combinar backend y frontend en un mismo proyecto
- hacer una infraestructura demasiado pesada
- introducir Docker si el objetivo es simplemente enseñar CRUD y arquitectura

12. Recomendación final

La mejor elección para una demo de clase es:

- Backend: .NET 10 + ASP.NET Core Web API
- ORM: Entity Framework Core
- Base de datos: SQLite
- Frontend: React + Vite + TypeScript

Esto ofrece una combinación muy equilibrada entre:

- claridad pedagógica
- modernidad tecnológica
- facilidad de ejecución local
- buena separación de capas
- alta legibilidad en una demo de clase

13. Propuesta de enunciado para la clase

> *Desarrollar una aplicación de gestión de tareas con arquitectura cliente-servidor. El backend se implementará en .NET 10 con ASP.NET Core y Entity Framework Core, utilizando SQLite como base de datos local. El frontend se implementará en React con Vite y TypeScript para consumir la API REST y permitir crear, listar, editar y eliminar tareas. El objetivo es demostrar el flujo completo entre interfaz, backend y persistencia de datos.*

14. Conclusión

Es una combinación sólida, moderna y muy adecuada para una demo de clase. Tiene la ventaja de que permite enseñar de forma clara conceptos esenciales de desarrollo full stack sin introducir demasiada complejidad operativa.

Si se mantiene simple, bien estructurada y bien explicada, esta app puede convertirse en una excelente demostración de:

- API REST
- acceso a base de datos
- ORM
- consumo de servicios desde frontend
- buenas prácticas de diseño de software

15. Siguiente paso recomendado

La siguiente fase ideal sería:

1. crear la estructura del proyecto backend
2. definir la entidad TodoItem

3. configurar DbContext y SQLite
4. crear la API CRUD
5. crear el frontend React
6. conectar el formulario y la lista con la API
7. probar el flujo completo de creación, edición y eliminación

Este documento ofrece la base conceptual para la demo y puede servir como punto de partida para la implementación real del proyecto.